

SE / SSE Assignment

Ask

You must use **Cursor / Kiro / VS Code with Spec driven development** with the below workflow

Requirement

↓

Specification

↓

Plan / Tasks

↓

Implementation

↓

Testing

↓

Review

↓

Fix

Do not start by asking AI:

"Build the complete application."

Generic Artefacts

Regardless of the IDE you use, maintain some basic hygiene steering files. They must include at the very least

1. Java Springboot Guidelines

2. Testing guidelines
3. API Standards
4. Documentation Skills
5. Commands to review code, spec and generate tests

If you use cursor, the structure might look something like below

```
rules/  
  java-springboot.md  
  testing.md  
  api-standards.md  
skills/  
  documentation/  
commands/  
  review-code.md  
  review-spec.md  
  generate-tests.md
```

The exact structure can vary depending on the IDE you use. The important thing is to understand and demonstrate **reusable AI instructions across the project context**.

Spec Artefacts

Create specifications before implementation. Once you do, an example structure looks like

```
spec/  
├── requirements.md  
├── architecture.md  
├── data-model.md  
├── api-contract.md  
├── state-machine.md  
├── ui-flow.md  
└── test-strategy.md
```

Prompt History

Any prompt you give must be saved to a file. Extensions like [SpecStory](#) allow you to do so for Cursor and VSCode out of the box. If you are using Kiro, create a small skill that records every prompt to a file.

Repository should contain:

```
.specstory/  
  history/
```

and:

```
docs/  
  prompt-history.md
```

Important

You must identify at least **meaningful mistakes or incorrect suggestions by the AI**.

This demonstrates that you are **using AI as an engineering assistant, not blindly accepting AI output**.

Token Optimisation

Use plugins like Graphify, Caveman, Codebase-memory MCP to optimise your token usage and ensure your tokens are efficiently utilised.

Exercise

Build a **Support Ticket Management System** using:

- Java 21
- Spring Boot
- PostgreSQL/H2
- REST API
- React/Next.js or equivalent frontend
- Cursor
- GitHub Copilot

The application is important, but **the main assessment is how you build it using AI.**

The main goal is to enable the team to be able to understand

- How to analyse requirements, create specifications
- How to use Spec driven development using Cursor / GitHub Copilot / Kiro
- How to manage AI context and validate AI-generated code
- How you test and debug

Application Requirements

Build a Support Ticket Management System with the following features:

1. Create a ticket.
2. List tickets.
3. View ticket details.
4. Update title, description, priority and assignee.
5. Add comments.
6. Search tickets by keyword.
7. Filter tickets by status.
8. Persist data in a database.
9. Validate input at the backend.
10. Display meaningful errors in the UI.

The following state machine must be enforced by the backend:

OPEN → IN_PROGRESS → RESOLVED → CLOSED

OPEN → CANCELLED
IN_PROGRESS → CANCELLED

Invalid transitions must be rejected.

Example:

CLOSED → OPEN ✗
RESOLVED → OPEN ✗
CANCELLED → OPEN ✗

Core Acceptance Criteria

The solution is complete when:

- Ticket can be created from UI.
- Tickets can be listed.
- Ticket details can be viewed.
- Ticket fields can be updated.
- Assignee can be changed.
- Comments can be added.
- Search works.
- Status filter works.
- Valid status transitions work.
- Invalid status transitions are rejected by backend.
- Data survives application restart.
- Backend validation works.
- UI shows meaningful errors.
- State-machine integration tests pass.
- No secrets are committed.